MACHINE LEARNING KURSI

# ML kodlar qo‘llanmasi

Jurat Shakarboev

1–11-darslar va oraliq nazorat asosida tuzilgan amaliy kodlar to‘plami

# Mundarija


**1. Kutubxona va import** 5

**2. Python asoslari** 5

2.1. O‘zgaruvchi va print 5

2.2. Satrdan harf olish (indeks) 6

2.3. Qoldiq amali % 6

2.4. if / elif / else 6

2.5. and va or 7

2.6. for va range 7

2.7. Ro‘yxat bo‘yicha sikl 7

2.8. Satrdagi harflar va ichma-ich for 8

**3. Pandas: ma’lumotni o‘qish va ko‘rish** 8

3.1. pd.read_csv — CSV faylni ochish 8

3.2. head va tail — boshidan yoki oxiridan qatorlar 9

3.3. info — ustunlar, turlar va bo‘sh joylar haqida qisqa ma’lumot 9

3.4. describe — sonli ustunlarning statistikasi 9

3.5. shape — qator va ustun soni 9

**4. Pandas: ustun va qatorni tanlash** 10

4.1. Bitta yoki bir nechta ustun 10

4.2. df.columns — ustun nomlari 10

4.3. select_dtypes — faqat sonli (yoki boshqa turdagi) ustunlar 10

4.4. pd.api.types.is_numeric_dtype — ustun sonlimi? 10

4.5. isin — qiymat ro‘yxat ichidami? 11

4.6. Shart bilan qator tanlash va astype 11

4.7. round — yaxlitlash 11

4.8. loc — qator va ustunni aniq manzil bilan olish 11

4.9. copy — jadvalning nusxasi 11

**5. Pandas: unikal qiymatlar** 12

5.1. unique — takrorlanmas qiymatlar ro‘yxati 12

5.2. nunique — nechta turli qiymat bor? 12

**6. Pandas: bo‘sh qiymatlarni topish** 12

6.1. isnull().sum() — har ustunda nechta bo‘sh joy 12

6.2. isnull().any() — shu ustunda umuman bo‘sh joy bormi? 13

6.3. isnull().sum().sum() — jami bo‘sh joy soni 13

6.4. pd.isnull — bitta qiymat bo‘shmi? 13

**7. Pandas: bo‘sh joyni to‘ldirish** 13

7.1. fillna + mean — o‘rtacha bilan 13

7.2. fillna + median — o‘rtadagi qiymat bilan 14

7.3. fillna + mode()[0] — eng ko‘p takrorlangan qiymat bilan 14

7.4. fillna + fixed — o‘zingiz belgilagan qiymat 14

7.5. for bilan avtomatik to‘ldirish 14

7.6. ffill — oldingi qiymat bilan to‘ldirish 15

**8. Pandas: o‘chirish** 15

8.1. drop — ustun (yoki qator) ni tashlab yuborish 15

8.2. dropna — bo‘sh joyi bor qatorlarni o‘chirish 15

**9. Encoding — matnni songa aylantirish** 16

9.1. pd.get_dummies — one-hot encoding 16

9.2. LabelEncoder — har kategoriyaga bitta butun son 16

**10. Scaling — sonlarni bir xil o‘lchamga keltirish** 17

10.1. StandardScaler — o‘rtacha 0, og‘ish 1 17

10.2. MinMaxScaler — odatda 0 dan 1 gacha 17

10.3. RobustScaler — median va IQR bilan 18

10.4. for bilan barcha sonli ustunlarni scale qilish 18

10.5. fit_transform va transform farqi 18

**11. Ma’lumotni modelga tayyorlash** 19

11.1. X va y ni ajratish 19

11.2. train_test_split — o‘qitish va sinash qismiga bo‘lish 19

**12. Linear Regression — sonni bashorat qilish** 19

**13. Logistic Regression — sinfga ajratish** 20

**14. Ko‘p uchraydigan xatolar** 21

14.1. could not convert string to float 21

14.2. inconsistent numbers of samples 21

14.3. fit_transform() missing 1 required positional argument: 'X' 21

14.4. Unknown label type: continuous 22

14.5. ChainedAssignmentError (fillna(..., inplace=True)) 22

14.6. ConvergenceWarning (Logistic Regression) 22

14.7. Yozuv xatosi encodingda ishlamasligi 22

**Talaba:** Jurat Shakarboev
**Manba:** 1–11-darslar va 6-mavzu (Oraliq nazorat) dagi daftarlar hamda darsliklar
**Maqsad:** darslarda haqiqatda uchragan buyruq va funksiyalarni oddiy tilda tushuntirish

Bu qo‘llanmada faqat sizning daftar va darsliklaringizda bo‘lgan kodlar bor. Misollar shu darslardagi dataset va ustun nomlari bilan yozilgan. Ba’zi joylarda kod biroz tozalab, bitta katakda ishlaydigan qilib moslashtirilgan.

---

## 1. Kutubxona va import

Darslarda asosan **pandas** va **scikit-learn** ishlatilgan. Jupyter daftarda paket o‘rnatish uchun quyidagi buyruq yozilgan (bu oddiy Python emas, notebook buyrug‘i):

NOTEBOOK

```
!pip install pandas
```

*Qayerda:* 5-dars, 6-mavzu, 8-dars.

Kerakli kutubxonani dastur boshida chaqirish:

PYTHON

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler, MinMaxScaler, RobustScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.metrics import accuracy_score, classification_report
```

`import` — kutubxonani ishga tushiradi. `as pd` — qisqa nom beradi, shuning uchun keyin `pandas.read_csv` o‘rniga `pd.read_csv` yoziladi. `from ... import ...` — kutubxonaning faqat kerakli qismini oladi.

*Qayerda:* deyarli barcha amaliy darslar (3–11-dars). 9-darsda `DecisionTreeClassifier` va `RandomForestRegressor` ham import qilingan, lekin daftarda `fit` qilinmagan.

---

## 2. Python asoslari

### 2.1. O‘zgaruvchi va `print`

```
s = "Uzbekistan"
a = 10
b = 4
print(a)
```

O‘zgaruvchi — qiymat saqlaydigan nom. `print` natijani ekranga chiqaradi.

*Qayerda:* 2-dars, 3-dars, 7-dars.

### 2.2. Satrdan harf olish (indeks)

```
s = "Uzbekistan"
print(s[0])    # birinchi harf: U
print(s[-1])   # oxirgi harf: n
print(s[-2])   # oxiridan ikkinchi harf: a
```

Qavslardagi raqam — harfning o‘rni. `0` birinchi belgi, `-1` oxirgi belgi.

*Qayerda:* 2-dars ( `2dars.ipynb` ), 6-mavzu ( `nazorat.ipynb` ).

### 2.3. Qoldiq amali `%`

```
a = 10
b = 4
print(a % b)   # 2, chunki 10 ni 4 ga bo‘lganda 2 qoldiq qoladi
```

`%` bo‘lishdan qolgan qoldiqni beradi. Juft-toqni tekshirishda ham ishlatiladi: `son % 2 == 0` juft demak.

*Qayerda:* 2-dars, 6-mavzu, 7-dars.

### 2.4. `if` / `elif` / `else`

```
ball = 80

if ball < 0 or ball > 100:
    print("xatolik: ball 0 dan 100 gacha bolishi kerak")
elif ball >= 86:
    print("alo - imtihondan otdi")
elif ball >= 71:
    print("yaxshi - imtihondan otdi")
elif ball >= 60:
    print("ortacha - imtihondan otdi")
else:
    print("yomon - imtihondan otmadi")
```

`if` — shart to‘g‘ri bo‘lsa, shu blok ishlaydi. `elif` — oldingisi to‘g‘ri bo‘lmasa, keyingi shartni tekshiradi. `else` — hech biri mos kelmasa ishlaydi.

*Qayerda:* 2-dars, 3-dars (baho, pasport, ob-havo, login, svetofor), 6-mavzu ( `nazorat.ipynb` ).

## 2.5. `and` va `or`

```
login = "Jurat"
parol = "Jurat1"

if login == "Jurat" and parol == "Jurat19":
    print("Xush kelibsiz!")
elif login == "Jurat":
    print("parol notogri")
else:
    print("bunday foydalanuvchi yoq")
```

`and` — ikkala shart ham to‘g‘ri bo‘lishi kerak. `or` — kamida bittasi to‘g‘ri bo‘lsa yetadi. `==` — tengligini tekshiradi.

*Qayerda:* 2-dars ( `if a > b or b < a` ), 3-dars (login), 6-mavzu.

## 2.6. `for` va `range`

```
for i in range(2, 11):
    if i % 2 == 0:
        print(i)
```

`for` — amalni qayta-qayta bajaradi. `range(2, 11)` 2 dan 10 gacha sonlar beradi (oxirgi son kirmaydi). Umumiy ko‘rinish: `range(start, stop, step)` .

Boshqa mashqlar: yig‘indi, ko‘paytirish jadvali, kvadrat:

```
yigindi = 0
for i in range(1, 11):
    yigindi = yigindi + i
print("1 dan 10 gacha bolgan sonlar yigindisi:", yigindi, "ga teng")

for i in range(1, 11):
    print("5 *", i, "=", 5 * i)

for i in range(1, 11):
    print(i, "ning kvadrati:", i * i)
```

*Qayerda:* 7-dars ( `7_dars.ipynb` ). 7-dars PDF da ham `for i in range` tushuntirilgan.

## 2.7. Ro‘yxat bo‘yicha sikl

```
sonlar = [4, 19, 7, 33, 2]
eng_katta = sonlar[0]
for son in sonlar:
    if son > eng_katta:
        eng_katta = son
print("Eng katta son:", eng_katta)
```

Ro‘yxatdagi har bir elementni birma-bir oladi va solishtiradi. 7-darsda mashina narxlari ro‘yxati ham shu usulda arzon-qimmatga ajratilgan.

*Qayerda:* 7-dars.

### 2.8. Satrdagi harflar va ichma-ich `for`

```
for harf in "Python":
    print(harf)

for i in range(1, 6):
    print("*" * i)
```

Birinchi kod so‘zdagi har bir harfni chiqaradi. Ikkinchisi yulduzchalardan uchburchak yasaydi: `*` ni `i` marta takrorlaydi.

*Qayerda:* 7-dars.

---

## 3. Pandas: ma’lumotni o‘qish va ko‘rish

Pandas — jadval (dataset) bilan ishlash kutubxonasi. Darslarda asosiy to‘plamlar: `upcoming-ultra-trails.csv`, `bank.csv`, `fruits_dataset.csv`, `automobile.csv`, `buyurtmalar_mashq.csv`, `uy_narxlari_regression.csv`.

### 3.1. `pd.read_csv` — CSV faylni ochish

PYTHON

```
import pandas as pd

df = pd.read_csv("buyurtmalar_mashq.csv")
```

Fayldagi jadvalni `df` (DataFrame) ga yuklaydi. Shundan keyin ustun va qatorlar bilan ishlash mumkin.

*Qayerda:* 3–11-darslar.

Qo‘shimcha variantlar:

```
df = pd.read_csv("upcoming-ultra-trails.csv", parse_dates=["event_date", "scraped_at"])

df = pd.read_csv("automobile.csv", na_values="?")
```

`parse_dates` — ko‘rsatilgan ustunlarni sana deb o‘qiydi (3-dars README). `na_values="?"` — fayldagi `?` belgisini bo‘sh qiymat ( `NaN` ) deb qabul qiladi (7-dars).

### 3.2. `head` va `tail` — boshidan yoki oxiridan qatorlar

```python
df.head()
df.head(5)
df.tail(5)
```

`head()` birinchi 5 qatorni, `head(5)` ham shuni, `tail(5)` oxirgi 5 qatorni ko‘rsatadi. Jadval qanday ko‘rinishini tezda ko‘rish uchun kerak.

*Qayerda:* 3-dars, 4-dars, 5-dars, 8–11-darslar.

### 3.3. `info` — ustunlar, turlar va bo‘sh joylar haqida qisqa ma’lumot

```python
df.info()
```

Nechta qator va ustun borligi, har bir ustunning turi (`int64`, `float64`, `str`) va nechtasida qiymat borligini chiqaradi.

*Qayerda:* 3–6, 8–11-darslar. 4-dars PDF da ham `df.info` tushuntirilgan. Qavslarni unutmaslik kerak: `df.info` (qavssiz) faqat funksiyaning o‘zini ko‘rsatadi, natija chiqarmaydi (6-mavzuda shunday bo‘lgan).

### 3.4. `describe` — sonli ustunlarning statistikasi

```python
df.describe()
```

Sonli ustunlar uchun soni, o‘rtacha, standart og‘ish, min, max va foizlarni (25%, 50%, 75%) ko‘rsatadi.

*Qayerda:* 5-dars (`vazifa.ipynb`, `automobile.csv`).

### 3.5. `shape` — qator va ustun soni

```python
print(df.shape)
x_train.shape, x_test.shape
```

Masalan, `(40000, 14)` — 40 000 qator va 14 ustun. Modelga bo‘lgandan keyin train/test hajmini tekshirish uchun ham ishlatilgan.

*Qayerda:* 7-dars, 8-dars (`vazifa.ipynb`), 11-dars.

---

# 4. Pandas: ustun va qatorni tanlash

## 4.1. Bitta yoki bir nechta ustun

```
df["name"]
df[["name", "locality", "country"]]
```

Bitta qavs — bitta ustun (Series). Ikki qavs — bir nechta ustunli yangi jadval.

*Qayerda:* 3-dars, 4-dars ( `df['education']` ).

## 4.2. `df.columns` — ustun nomlari

PYTHON

```
df.columns
```

Jadvaldagi barcha ustun nomlarini beradi. `for col in df.columns` bilan har bir ustunni aylanib chiqish mumkin.

*Qayerda:* 7-dars PDF, 8-dars ( `for col in df.columns` ).

## 4.3. `select_dtypes` — faqat sonli (yoki boshqa turdagi) ustunlar

PYTHON

```
num_col = df.select_dtypes(include=["int64", "float64"]).columns
num_cols = df.select_dtypes(include="number").columns
```

Faqat raqamli ustunlarni ajratib oladi. Scaling yoki `fillna(mean)` dan oldin foydali, chunki matnli ustunni o‘rtacha bilan to‘ldirib bo‘lmaydi.

*Qayerda:* 8-dars, 9-dars, 10-dars.

## 4.4. `pd.api.types.is_numeric_dtype` — ustun sonlimi?

PYTHON

```
import pandas as pd

for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        print(col, "- sonli ustun")
```

Ustun turi son bo‘lsa `True` qaytaradi. 8–9-darslarda bo‘sh joyni to‘ldirish yoki scaling qilishdan oldin shu tekshiruv ishlatilgan.

*Qayerda:* 8-dars, 8-dars vazifa, 9-dars.

### 4.5. `isin` — qiymat ro‘yxat ichidami?

```
df = df[df["holat"].isin(["yetkazildi", "qaytarildi", "bekor_qilindi"])]
```

Faqat `holat` shu uch qiymatdan biri bo‘lgan qatorlarni qoldiradi. 9-dars izohida yozilganidek, `isin` — “ichida bormi” degani.

*Qayerda:* 9-dars.

### 4.6. Shart bilan qator tanlash va `astype`

```
df["target"] = (df["holat"] == "yetkazildi").astype(int)
```

`holat` “yetkazildi” bo‘lsa 1, bo‘lmasa 0 yoziladi. `astype(int)` turini butun songa o‘tkazadi. Logistic regression uchun nishon (y) shunday diskret qilinadi.

*Qayerda:* 9-dars. 11-darsda shunga o‘xshash: `y = df["chegirma_foiz"].round().astype(int)`.

### 4.7. `round` — yaxlitlash

```
y = df["chegirma_foiz"].round().astype(int)
```

Kasr sonni eng yaqin butun songa yaxlitlaydi, keyin `int` qiladi. Logistic modelga uzluksiz (kasr) nishon bermaslik uchun ishlatilgan.

*Qayerda:* 11-dars.

### 4.8. `loc` — qator va ustunni aniq manzil bilan olish

```
ortacha = a["bore"].mean()
for i in range(len(a)):
    if pd.isnull(a.loc[i, "bore"]):
        a.loc[i, "bore"] = ortacha
print("Mean:", a["bore"].isnull().sum(), "ta bosh joy qoldi")
```

`a.loc[i, "bore"]` — `i`-qatordagi `bore` katagi. `pd.isnull` shu katak bo‘shligini tekshiradi. `len(a)` — qatorlar soni.

*Qayerda:* 7-dars (mean, median, mode va fixed ni `for` + `loc` bilan to‘ldirish).

### 4.9. `copy` — jadvalning nusxasi

PYTHON

```
a = df.copy()
toza = df.copy()
```

Asl `df` ni buzmasdan, nusxada sinab ko‘rish uchun. 8-dars vazifasida mean, median, mode, drop usullari shu bilan alohida solishtirilgan.

*Qayerda:* 8-dars ( `vazifa.ipynb` ).

---

## 5. Pandas: unikal qiymatlar

### 5.1. `unique` — takrorlanmas qiymatlar ro‘yxati

```
df["name"].unique()
df["event_name"].unique()
```

Ustundagi barcha turli qiymatlarni ko‘rsatadi (takrori yo‘q).

*Qayerda:* 3-dars.

### 5.2. `nunique` — nechta turli qiymat bor?

PYTHON

```
df.nunique()
df["locality"].nunique()
df["chegirma_foiz"].nunique()
```

Har bir ustunda (yoki bitta ustunda) nechta unikal qiymat borligini sanaydi. 3-dars izohida “ustundagi classlar soni” deb yozilgan. `nunique` ham qavs bilan chaqiriladi.

*Qayerda:* 3-dars, 5-dars, 11-dars. 10-darsda encoding oldidan `if df[col].nunique() ⩽ 5` sharti ham bor.

---

## 6. Pandas: bo‘sh qiymatlarni topish

### 6.1. `isnull().sum()` — har ustunda nechta bo‘sh joy

PYTHON

```
df.isnull().sum()
```

Har bir ustundagi bo‘sh ( `NaN` ) kataklar sonini chiqaradi. Tozalashdan oldin va keyin tekshirish uchun asosiy vosita.

*Qayerda:* 3–8-darslar, 11-dars. 4-dars PDF da `df.isnull` tushuntirilgan.

### 6.2. `isnull().any()` — shu ustunda umuman bo‘sh joy bormi?

```python
df.isnull().any()
```

Har ustun uchun `True` (bor) yoki `False` (yo‘q) qaytaradi. 8-darsda `if df[col].isnull().any()` bilan faqat bo‘sh joyi bor ustunlar to‘ldirilgan.

*Qayerda:* 3-dars, 8-dars.

### 6.3. `isnull().sum().sum()` — jami bo‘sh joy soni

```python
df.isnull().sum().sum()
```

Butun jadvaldagi bo‘sh kataklarning umumiy soni. 11-darsda tozalashdan keyin `0` chiqqani shu bilan tekshirilgan.

*Qayerda:* 8-dars vazifa, 11-dars.

### 6.4. `pd.isnull` — bitta qiymat bo‘shmi?

```
if pd.isnull(a.loc[i, "bore"]):
    a.loc[i, "bore"] = ortacha
```

Bitta katakning bo‘shligini tekshiradi. Sikl ichida ishlatilgan.

*Qayerda:* 7-dars.

---

## 7. Pandas: bo‘sh joyni to‘ldirish

### 7.1. `fillna` + `mean` — o‘rtacha bilan

```
df["normalized-losses"] = df["normalized-losses"].fillna(df["normalized-losses"].mean())
```

Bo‘sh kataklarga shu ustunning o‘rtacha qiymati yoziladi. Asosan sonli ustunlar uchun.

*Qayerda:* 4-dars ( `age` ), 5-dars vazifa, 6-mavzu, 7-dars, 8-dars. 4-darsda `inplace=True` varianti ham bor, lekin u ogohlantirish bergan (qarang: oxirgi bo‘lim).

Butun jadvaldagi sonli ustunlarni bir yo‘la to‘ldirish (11-dars):

```python
df = df.fillna(df.mean(numeric_only=True))
```

`numeric_only=True` — faqat sonli ustunlarni hisoblaydi, matnga teginmaydi.

### 7.2. `fillna` + `median` — o‘rtadagi qiymat bilan

```
df["balance"] = df["balance"].fillna(df["balance"].median())
```

Sonlarni tartiblaganda o‘rtada turgan qiymat qo‘yiladi. Chet qiymatlarga (outlier) o‘rtachaga qaraganda chidamliroq.

*Qayerda:* 4-dars, 7-dars, 9-dars. 6-mavzuda `bore` matn bo‘lgani uchun `median()` xato bergan.

### 7.3. `fillna` + `mode()[0]` — eng ko‘p takrorlangan qiymat bilan

```
df["num-of-doors"] = df["num-of-doors"].fillna(df["num-of-doors"].mode()[0])
df["job"] = df["job"].fillna(df["job"].mode()[0])
```

`mode()` eng ko‘p uchraydigan qiymat(lar)ni beradi. `[0]` — shulardan birinchisini oladi. Matnli ustunlar ( `job` , `num-of-doors` , `viloyat` ) uchun qulay.

*Qayerda:* 4-dars, 5-dars vazifa, 6-mavzu, 7-dars, 8–9-darslar. 4-dars va 7-dars PDF da `mode()[0]` alohida eslatilgan.

### 7.4. `fillna` + fixed — o‘zingiz belgilagan qiymat

```
df["bore"] = df["bore"].fillna(15)
a["horsepower"] = a["horsepower"].fillna(0)
a[col] = a[col].fillna("Nomalum")
```

Bo‘sh joyni aniq bir qiymat bilan to‘ldiradi: son uchun `0` yoki `15` , matn uchun `"Nomalum"` .

*Qayerda:* 4-dars ( `'Jurat'` ), 6-mavzu, 7-dars, 8-dars vazifa.

Bir nechta ustunni lug‘at bilan to‘ldirish:

```
b = b.fillna({"horsepower": 0, "peak-rpm": 0, "stroke": 0})
```

*Qayerda:* 7-dars.

### 7.5. `for` bilan avtomatik to‘ldirish

PYTHON

```
for col in df.columns:
    if df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(df[col]):
            df[col] = df[col].fillna(df[col].mean())
        else:
            df[col] = df[col].fillna(df[col].mode()[0])
```

Har bir ustunni tekshiradi: son bo‘lsa mean, matn bo‘lsa mode qo‘yadi.

*Qayerda:* 8-dars, 9-dars (9-darsda matn keyin `LabelEncoder` ham qilingan).

### 7.6. `ffill` — oldingi qiymat bilan to‘ldirish

```python
df = df.ffill(axis=1)
```

Bo‘sh katakni o‘sha qatordagi oldingi (chapdagi) qiymat bilan to‘ldiradi. `axis=1` — qator bo‘ylab.

*Qayerda:* 6-mavzu ( `ML_boyicha.ipynb` ).

---

## 8. Pandas: o‘chirish

### 8.1. `drop` — ustun (yoki qator) ni tashlab yuborish

```python
df = df.drop(columns=["buyurtma_id", "mijoz_id", "sana", "mahsulot"])
df = df.drop("education", axis=1)
df = df.drop(columns=["bore"])
```

Keraksiz ustunlarni olib tashlaydi. `axis=1` — ustun, `axis=0` — qator. Natijani `df = ...` ga qayta yozish kerak, aks holda asl jadval o‘zgarmaydi.

*Qayerda:* 4-dars, 5-dars, 6-mavzu, 9-dars, 10-dars, 11-dars. 4-dars PDF da ham `axis` va `inplace` tushuntirilgan.

### 8.2. `dropna` — bo‘sh joyi bor qatorlarni o‘chirish

```python
df = df.dropna(subset=["jami_summa_uzs"])
df = df.dropna()
```

`subset` — faqat shu ustunda bo‘sh joy bo‘lsa, o‘sha qator o‘chadi (11-dars: nishon `jami_summa_uzs` bo‘sh bo‘lmasin). `dropna()` — istalgan ustunda bo‘sh joyi bor qatorni o‘chiradi. 8-dars vazifasida bu usul 20 893 qatorni olib tashlagan.

*Qayerda:* 4-dars, 5-dars vazifa, 8-dars vazifa, 11-dars.

---

## 9. Encoding — matnni songa aylantirish

Model odatda matnni tushunmaydi, shuning uchun kategoriya ustunlari songa o‘tkaziladi. 5- va 7-dars PDF da frequency, target va ordinal encoding ham tushuntirilgan, lekin daftarlarda ularning tayyor kodi yo‘q. Amalda ishlatilganlari quyidagilar.

### 9.1. `pd.get_dummies` — one-hot encoding

```python
import pandas as pd

dummies = pd.get_dummies(df["label"], prefix="ustun", dtype=int)
df = pd.concat([df.drop(columns=["label"]), dummies], axis=1)
```

Har bir kategoriya uchun alohida 0/1 ustun ochiladi. Masalan, `label` dagi `olma`, `banan` dan `ustun_olma`, `ustun_banan` chiqadi. `prefix` — yangi ustun nomining boshi. `dtype=int` — natija 0 va 1 (son) bo‘lsin. `concat` — eski jadval va yangi ustunlarni yonma-yon birlashtiradi (`axis=1`).

*Qayerda:* 5-dars (`fruits_dataset.csv`, `label`); 5-dars vazifa (`automobile.csv`, `fuel-system`, `prefix="system"`).

Bir nechta ustunni bir yo‘la, birinchi kategoriyani tashlab:

```python
df = pd.get_dummies(
    df,
    columns=["viloyat", "kategoriya", "tolov_usuli", "holat"],
    drop_first=True,
)
```

`drop_first=True` — har guruhdan bitta ustunni tashlaydi (takroriy axborotni kamaytirish uchun).

*Qayerda:* 11-dars. 10-darsda ham `get_dummies` yozilgan, lekin `"object"` o‘rniga `"obejct"` yozilgani uchun u katak ishlamagan.

### 9.2. `LabelEncoder` — har kategoriyaga bitta butun son

```python
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
df["label"] = encoder.fit_transform(df["label"])
```

Masalan, `uzum` → 4, `tarvuz` → 3, `banan` → 1. Bitta ustun saqlanadi, lekin sonlar orasida “tartib” bordek ko‘rinishi mumkin.

Sikl bilan bir nechta matnli ustun:

```python
from sklearn.preprocessing import LabelEncoder

for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = df[col].fillna(df[col].median())
    else:
        df[col] = df[col].fillna(df[col].mode()[0])
        df[col] = LabelEncoder().fit_transform(df[col])
```

*Qayerda:* 5-dars ( `label-encoding.ipynb` ), 9-dars, 10-dars (import qilingan).

---

## 10. Scaling — sonlarni bir xil o‘lchamga keltirish

Juda katta va juda kichik sonlarni model uchun muvozanatlashtiradi. 8-dars mavzusi.

### 10.1. `StandardScaler` — o‘rtacha 0, og‘ish 1

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df["miqdor"] = scaler.fit_transform(df[["miqdor"]])
```

Qiymatlar o‘rtacha atrofida, standart og‘ishi 1 bo‘ladigan shaklga o‘tadi. Diqqat: `StandardScaler()` — qavs kerak, aks holda xato chiqadi. `df[["miqdor"]]` — ikki qavs (jadval ko‘rinishi); bitta qavs xato berishi mumkin.

*Qayerda:* 8-dars, 8-dars vazifa, 9-dars.

### 10.2. `MinMaxScaler` — odatda 0 dan 1 gacha

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df["baho"] = scaler.fit_transform(df[["baho"]])
```

Eng kichik qiymat 0 ga, eng katta 1 ga yaqinlashadi. 10-darsda ham bir nechta ustun shu scaler bilan o‘zgartirilgan.

*Qayerda:* 8-dars, 9-dars, 10-dars.

### 10.3. `RobustScaler` — median va IQR bilan

```python
from sklearn.preprocessing import RobustScaler

scaler = RobustScaler()
df["chegirma_foiz"] = scaler.fit_transform(df[["chegirma_foiz"]])
```

Median va kvartillar oralig‘iga qarab masshtablaydi. 8-dars vazifasidagi izoh: outlier (juda chetdagi qiymat) larga chidamliroq.

*Qayerda:* 8-dars, 8-dars vazifa.

### 10.4. `for` bilan barcha sonli ustunlarni scale qilish

```python
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
for col in df.columns:
    if pd.api.types.is_numeric_dtype(df[col]):
        df[col] = scaler.fit_transform(df[[col]])
```

Har bir sonli ustunni birma-bir 0–1 oralig‘iga keltiradi.

*Qayerda:* 8-dars, 8-dars vazifa, 9-dars ( `select_dtypes` bilan).

### 10.5. `fit_transform` va `transform` farqi

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

`fit_transform` — avval o‘rganadi (masalan, o‘rtacha va og‘ishni hisoblaydi), keyin o‘zgartiradi. Buni **faqat train** da qilish to‘g‘ri. `transform` — o‘sha o‘rganilgan qoidani testga qo‘llaydi, qayta o‘rganmaydi. Testga ham `fit_transform` qilinsa, test ma’lumoti “o‘qib yuboriladi”.

*Qayerda:* `fit_transform` — 5, 8, 9, 10-darslar; `transform` alohida — 9-dars (train/test scaling).

---

# 11. Ma’lumotni modelga tayyorlash

## 11.1. `X` va `y` ni ajratish

```python
X = df.drop(columns=["jami_summa_uzs"])
y = df["jami_summa_uzs"]
```

`X` — modelga beriladigan tushuntiruvchi ustunlar (kirish). `y` — bashorat qilinadigan nishon (chiqish). `drop` nishonni `X` dan olib tashlaydi, lekin asl `df` ni o‘zgartirmaydi (bu yerda `inplace` yo‘q).

*Qayerda:* 9-dars ( `target` ), 10-dars ( `markazgacha_km` ), 11-dars ( `jami_summa_uzs` , keyin `chegirma_foiz` ).

## 11.2. `train_test_split` — o‘qitish va sinash qismiga bo‘lish

```python
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Ma’lumotning 80 foizi o‘qitish ( `train` ), 20 foizi sinash ( `test` ) uchun ajraladi. `random_state=42` — har safar bir xil tasodifiy bo‘linish bo‘lishi uchun. Bu qadamni tozalash va encoding **dan keyin** qilish kerak, aks holda qatorlar soni mos kelmaydi.

*Qayerda:* 9-dars, 10-dars, 11-dars.

---

# 12. Linear Regression — sonni bashorat qilish

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

lr = LinearRegression()
lr.fit(x_train, y_train)
y_pred = lr.predict(x_test)

print("R2:", r2_score(y_test, y_pred))
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
```

- `LinearRegression()` — chiziqli regressiya modelini yaratadi.
- `fit` — train ma’lumotidan chiziqni o‘rganadi.

- `predict` — test uchun taxminiy `y` ni hisoblaydi.
- `r2_score` ($R^2$) — model ma’lumotdagi o‘zgarishning qancha qismini tushuntirishini ko‘rsatadi (1 ga yaqin — yaxshi; manfiy — oddiy o‘rtachadan ham yomon).
- `mean_absolute_error` (MAE) — xatoning o‘rtacha mutlaq qiymati.
- `mean_squared_error` (MSE) — xato kvadratlarining o‘rtachasi.

11-darsda `jami_summa_uzs` ni bashorat qilganda $R^2$ taxminan 0,76 chiqqan. 10-darsda boshqa nishon ( `markazgacha_km` ) bilan $R^2$ manfiy bo‘lgan.

*Qayerda:* 10-dars, 11-dars. 9-darsda `LinearRegression` faqat import qilingan.

---

## 13. Logistic Regression — sinfga ajratish

PYTHON

```python
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

log_reg = LogisticRegression()
log_reg.fit(x_train, y_train)
y_pred = log_reg.predict(x_test)

score = accuracy_score(y_test, y_pred)
print(score)
print(classification_report(y_test, y_pred))
```

Logistic regression javobni **sinflarga** ajratadi (masalan, yetkazildi / yetkazilmadi), uzluksiz narx hisoblamaydi. Shuning uchun `y` butun son yoki aniq sinf bo‘lishi kerak.

9-darsda muvozanat uchun qo‘shimcha parametr ishlatilgan:

```python
model = LogisticRegression(class_weight="balanced")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

`class_weight="balanced"` — kam uchraydigan sinfga ko‘proq e’tibor beradi. `accuracy_score` — to‘g‘ri topilgan ulush. `classification_report` — precision, recall, f1-score ni sinfma-sinf ko‘rsatadi.

9-darsda `holat == "yetkazildi"` nishoni bilan aniqlik taxminan 0,51; 11-darsda `chegirma_foiz` (yaxlitlab, `int` qilib) bilan taxminan 0,56 chiqqan.

*Qayerda:* 9-dars, 11-dars. 10-dars PDF da logistic (binary va multiclass) haqida qisqa qayd bor.

---

## 14. Ko‘p uchraydigan xatolar

Quyidagilar daftarlar va shu darslardagi amaliyotda ko‘rilgan xatolar. Sababi va yechimi qisqa yozilgan.

### 14.1. `could not convert string to float`

**Sabab:** model yoki hisob-kitobga matn tushib qolgan. 11-dars amaliyotida `buyurtma_id` kabi qiymat (`ORD-014308`) `float` ga aylantirilmagan. 6-mavzuda esa `bore` ustuni matn bo‘lgani uchun `median()` ishlamagan: `TypeError: Cannot perform reduction 'median' with string dtype`.

**Yechim:** ID, sana, mahsulot nomi kabi keraksiz matnli ustunlarni `drop` qiling. Qolgan kategoriyalarni `get_dummies` yoki `LabelEncoder` qiling. Son deb o‘ylangan ustun aslida `str` bo‘lsa, avval turini tekshiring (`df.info()`).

PYTHON

```python
df = df.drop(columns=["buyurtma_id", "mijoz_id", "sana", "mahsulot"])
df = pd.get_dummies(
    df,
    columns=["viloyat", "kategoriya", "tolov_usuli", "holat"],
    drop_first=True,
)
```

### 14.2. `inconsistent numbers of samples`

**Sabab:** `X` va `y` da qatorlar soni bir xil emas. Bu odatda `train_test_split` tozalashdan **oldin** chaqirilganda, yoki `X` katta harf, `x` kichik harf bilan ikki xil o‘zgaruvchi aralashganda chiqadi. Masalan, eski `x` da 40 000 qator, tozalangan `y` da 37 266 qator qolgan.

**Yechim:** avval `dropna`, `drop`, `fillna`, `get_dummies` ni tugating. Keyin bir xil jadvaldan `X` va `y` ni oling, shundan keyin `train_test_split` qiling. Bitta nomni tanlang (`X` yoki `x`) va hammasida shuni ishlating.

PYTHON

```python
df = df.dropna(subset=["jami_summa_uzs"])
X = df.drop(columns=["jami_summa_uzs"])
y = df["jami_summa_uzs"]
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

### 14.3. `fit_transform() missing 1 required positional argument: 'X'`

**Sabab:** scaler yaratilganda qavs unutilgan: `scaler = StandardScaler` (bu klassning o‘zi). Keyin `scaler.fit_transform(df[num_col])` ma’lumotni `X` deb emas, `self` deb qabul qiladi.

**Yechim:** obyekt yaratishda qavs qo‘ying.

```python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df["miqdor"] = scaler.fit_transform(df[["miqdor"]])
```

`df.info` va `df.nunique` ham qavssiz chaqirilsa, natija o‘rniga funksiya obyekti chiqadi. Ularga ham `()` kerak.

### 14.4. `Unknown label type: continuous`

**Sabab:** Logistic regression sinflarni kutadi, lekin `y` uzluksiz son (masalan, `jami_summa_uzs` yoki `fillna(mean)` dan keyingi `12.37` kabi `chegirma_foiz`).

**Yechim:** nishonni diskret qiling. Chegirma uchun:

```
y = df["chegirma_foiz"].round().astype(int)
```

Yoki klassifikatsiya uchun 9-darsdagi kabi aniq sinf yasang:

```
y = (df["holat"] == "yetkazildi").astype(int)
```

To‘g‘riroq yo‘l: `fillna` dan oldin nishoni bo‘sh qatorlarni `dropna(subset=[...])` bilan olib tashlash, shunda `y` ga sun’iy o‘rtacha aralashmaydi.

### 14.5. `ChainedAssignmentError` (`fillna(..., inplace=True)`)

**Sabab:** 4-darsda `df["age"].fillna(..., inplace=True)` yozilgan. Bu ba’zan asl jadvalni yangilamaydi va pandas ogohlantirish beradi.

**Yechim:** natijani qayta yo‘ling:

```
df["age"] = df["age"].fillna(df["age"].mean())
```

### 14.6. `ConvergenceWarning` (Logistic Regression)

**Sabab:** 11-darsda model 100 ta iteratsiyada yaqinlashmadi. Ko‘pincha ma’lumot scale qilinmagani yoki iteratsiya yetmagani tufayli bo‘ladi.

**Yechim:** 9-darsdagidek avval `StandardScaler` qo‘ying (`fit_transform` train ga, `transform` test ga). Kerak bo‘lsa, `LogisticRegression(max_iter=1000)` deb iteratsiyani oshiring (bu parametr 11-dars daftarida yo‘q, lekin xato matnida tavsiya qilingan).

### 14.7. Yozuv xatosi encodingda ishlamasligi

**Sabab:** 10-darsda `if df[col].dtype == "obejct"` yozilgan (`object` emas). Shuning uchun encoding kataklari ishlamagan.

**Yechim:** to‘g‘ri yozing: `"object"`. Ishlamayotgan kodni `df.info()` bilan tekshiring: matnli ustun hali ham `str` / `object` bo‘lib tursa, encoding o‘tmagan.

---

*Bu qo‘llanma faqat darslarda uchragan kodlar asosida tuzilgan. `value_counts`, `drop_duplicates`, `OneHotEncoder`, `coef_`, `intercept_` kabi buyruqlar daftarlarda bo‘lmagani uchun kiritilmagan.*